

Object Localization in 3D Semantic Maps via Cluster Bearing and Monocular Distance

Author Name

Affiliation

Organization

City, Country

email address

Abstract—Localizing objects in a 3D semantic map for indoor autonomous delivery robots is commonly solved using stereo cameras, RGB-D or LiDAR sensors, or camera-based triangulation methods that depend on the pose odometry of each frame. This paper presents an alternative approach that uses only a low-cost monocular camera (ESP32-CAM), where accuracy comes from algorithm design rather than hardware upgrades. The bearing of the target object is computed from a semantic point cluster — MapAnything reconstructs the point cloud, YOLO11n attaches the class label, and DBSCAN performs clustering — with the result transformed into real-world coordinates using the Umeyama algorithm, fitted once over the entire camera trajectory. Distance is estimated from a single frame using the YOLO bounding-box height together with basic geometric principles. Experiments were conducted over sixteen independent capture sessions at three ground-truth distances. At 3 m and 5 m the mean error stays under 2%, with standard deviations of 0.19 m and 0.22 m; at 7 m a systematic bias of about −11% appears, but the standard deviation is only 0.07 m, indicating that the error is dominated by a calibratable bias rather than random noise.

Index Terms—3D semantic map, object localization, MapAnything, YOLO, autonomous delivery robot, global fit Umeyama, monocular distance estimation

I. INTRODUCTION

The indoor autonomous delivery robot needs to construct and maintain a semantic 3D map to understand object localization and reference points— the object to be delivered, as well as fixed objects such as plant pots, tables, and chairs that can serve as reference points for navigation. A typical pipeline that combines three components : (i) a 2D detection model (YOLO) that runs on each frame, (ii) a monocular 3D reconstruction model, and (iii) a mechanism for determining the absolute position of objects in the real world, often depends on information from localizing odometry of the robot.

The real-world deployment with low-cost hardware and odometry that accumulates errors over time raises a fundamental question: what really determines the accuracy, and what can we do to improve confidence under limited resources? On the robot Hawkbot (ROS2 Jazzy) equipped with the camera ESP32-CAM, we realize that methodologies that depend on pose odometry of every single frame are sensitive to local noise. The paper presents another methodology: combining orientation, calculated from semantic clustering of objects in MapAnything, with distance estimated from a monocular

image, rather than relying on odometry. All sensors include only a low-cost monocular camera — it does not use stereo, RGB-D, or LiDAR; the accuracy mainly comes from algorithm design. The methodology is validated over sixteen independent sessions at three distances (3 m, 5 m, 7 m), showing stable and consistent results.

II. RELATED WORKS

The slam systems such as ORB-SLAM [1] optimize camera pose and structure through bundle adjustment and loop closure; the semantic SLAM systems such as SemanticFusion [2], Kimera [3] extend this approach by attaching class labels to the geometric map in real time. This requires online pose optimization [4], which is not suitable for the limited computational resources of edge devices used in this research. (Section III-B).

Detecting objects on YOLO [5], [6]; other methods such as Faster R-CNN [7] or DETR [8] can achieve higher accuracy, but are heavier. With 3D reconstruction using monocular camera images, these direct inference models such as (feed-forward) MapAnything [9], Depth Anything [10], and Mi-DaS [11] replace the traditional pipeline SfM/MVS with a single inference pass, without multiview camera calibration.

To transform between intrinsic space of the reconstruction model and real-world space from odometry, the Umeyama algorithm [12] is classical method for registering fixed point with unknown scale, it is often combined ICP [13] for local alignment; DBSCAN [14] clusters points in the map without requiring to know the number of clusters in advance, making it suitable for separating semantic classes from label noise (Algorithm 1).

Unlike the approaches above, the method used in the paper does not optimize pose online or perform triangulation for each frame; it separates orientation signals and distance (single-image) from pose odometry for each frame — it is suitable for the post-processing constraint and low-cost hardware of the Hawkbot base.

III. METHODOLOGY

A. Hardware

The entire system is built from popular hardware, low-cost (Table I) — Not using camera stereo, RGB-D sensor,

TABLE I: System configuration

Components	Configurations
Robot	Hawkbots, ROS2 [15] Jazzy (container hawkbot-bringup)
Camera	ESP32-CAM, QVGA resolution (320 × 240 px)
Odometry Processing	nav_msgs/Odometry, topic /odom remote GPU server, connected through a local network
Detection	YOLO11n [5], [6] (nano), trained on the COCO-80 dataset
3D reconstruction	MapAnything [9] (facebook/map-anything)

or LiDAR. This choice makes accuracy mainly depend on algorithm design, rather than on sensor quality.

Camera intrinsic matrix K (Calibration using checkerboard pattern, 74/75 valid images, with an average reprojection error of 0.12 pixel):

$$K = \begin{bmatrix} 428.72 & 0 & 148.61 \\ 0 & 427.60 & 126.02 \\ 0 & 0 & 1 \end{bmatrix}$$

B. Current hardware limitations

Camera resolution. The camera ESP32-CAM is configured at QVGA to ensure sufficient bandwidth for image transmission and a stable frame rate. This low resolution significantly limits the angular localization accuracy of any method based on pixel positions. When evaluating the use of the localization marker ArUco [16] 15 cm(Section V-B), a one-degree error in estimating the marker angle, at a distance of more than two meters, can correspond to a real-world positional error of up to several tens of centimeters.

Computational capacity. ESP32-CAM does not perform computer vision processing locally; all Yolo and MapAnything inference runs on a remote GPU server. The current pipeline operates offline in post-processing rather than as an online SLAM system; adding an online correction mechanism, such as loop closure and pose-graph optimization, requires computational resources the current platform cannot provide.

The two limitations above are reasons why this research focuses on finding a software methodology; it does not require upgrading hardware- the methodology, which combines semantic clusters and single image distance(Section III-D) — rather than adding more hardware- is proposed as an improvement in future development (Section V-C).

C. Pipeline Architecture

Three main processing block of pipeline: (i) **3D reconstruction** using MapAnything (S3), (ii) **Attaching semantic class label YOLO on 3D map** to have semantic point cloud (S4), and (iii) **Estimating single-image distance** (S7) — two signals (ii) and (iii) then is combined without triangulation through many frames.

S1. Collection: Robot moves independently; the camera continuously captures images (~ 1 frame/second), recording

localization/ odometry orientation for each frame into `poses.json`.

S2. Priority frame selection: Running YOLO on the overall original frame, selecting up to 130 frames containing the target object, and uniformly sampling across each visit.

S3. 3D reconstruction: Running MapAnything [9] on selected frames, reconstructing a pure geometric point cloud and the intrinsic camera trajectory.

S4. Attaching semantic class label (YOLO $\rightarrow$ 3D): Running YOLO on each frame, attaching COCO class label for each 3D point in bounding box (Algorithm 1), creates *semantic point cloud*.

S5. Global fit: Umeyama fit between MapAnything trajectory and odometry trajectory $\rightarrow (s, R, t)$, calculated once for overall session.

S6. Object orientation: Filtering point belonging to the target class label, DBSCAN clustering [14], Selecting max cluster center; apply global fit to inference the orientation.

S7. Object distance: on the only frame, Single-image estimation from the YOLO bounding box height, real height, and camera focal length (§ III-D).

S8. Combining: localization = reference camera localization + orientation $\times$ distance.

Algorithm 1 describes the mechanism for attaching semantic class labels for 3D points at S4 — basic for determining target clusters at S6.

Algorithm 1 Attaching semantic class labels for 3D point cloud

Require: Selected frame $\{I_1, \dots, I_n\}$; MapAnything $\mathcal{M}$; YOLO $\mathcal{Y}$
Ensure: labeled Point cloud $\mathcal{P} = \{(\mathbf{x}_j, l_j)\}$

```

1:  $\mathcal{P} \leftarrow \emptyset$ 
2: for  $i \leftarrow 1$  to  $n$  do
3:    $(D_i, K_i, T_i, M_i) \leftarrow \mathcal{M}(I_i)$   $\triangleright$  depth, K, pose, mask
4:    $\mathbf{P}_i \leftarrow \text{DEPTHToWORLD}(D_i, K_i, T_i)$ 
5:    $B_i \leftarrow \mathcal{Y}(I_i)$   $\triangleright (l, \text{bbox}, \text{conf})$ , Filtering by conf
6:   for each pixel  $(u, v)$ ,  $M_i(u, v) = \text{true}$  do
7:      $l \leftarrow -1$   $\triangleright$  default: background
8:     for each  $(l', \text{bbox}, \text{conf}) \in B_i$  do
9:       if  $(u, v) \in \text{bbox}$  then
10:         $l \leftarrow l'$ ; break
11:       end if
12:     end for
13:      $\mathcal{P} \leftarrow \mathcal{P} \cup \{(\mathbf{P}_i(u, v), l)\}$ 
14:   end for
15: end for
16: return  $\mathcal{P}$ 
```

Limitations of labeling mechanism. The check at line 8 only determines whether a pixel is inside the 2D YOLO bounding box, not semantic segmentation at the pixel level. Background points or other objects falling inside the bounding box may also be incorrectly labeled. This is why the DBSCAN step at S6 is necessary: the densest point cluster — corresponding to the actual object observed consistently from multiple views — is separated from scattered noise points.

D. Formula

Global fit (Umeyama fit) [12]. Let $\{\mathbf{q}_k\}$ denote camera positions from MapAnything (projected onto the horizontal

plane) and $\{\mathbf{p}_k\}$ denote odometry positions, respectively. The Umeyama algorithm finds $s \in \mathbb{R}$, $R \in SO(2)$, and $\mathbf{t}$ that minimize:

$$(s^*, R^*, \mathbf{t}^*) = \arg \min_{s, R, \mathbf{t}} \sum_k \|\mathbf{p}_k - (sR\mathbf{q}_k + \mathbf{t})\|^2 \quad (1)$$

It is solved using SVD on the cross-covariance matrix between the two centered point sets. The point $\mathbf{x}_{\text{raw}}$ in the MapAnything is transformed into real-world space as: $\mathbf{x}_{\text{real}} = s^*R^*\mathbf{x}_{\text{raw}} + \mathbf{t}^*$.

Orientation. With $\mathbf{c}_{\text{raw}}$ is centroid of the largest semantic point cluster and $\mathbf{o}$ is reference camera position:

$$\hat{\mathbf{b}} = \frac{(s^*R^*\mathbf{c}_{\text{raw}} + \mathbf{t}^*) - \mathbf{o}}{\|(s^*R^*\mathbf{c}_{\text{raw}} + \mathbf{t}^*) - \mathbf{o}\|} \quad (2)$$

Single-image distance. With bbox height h_{px} , real height H , focal length f_y :

$$d = \frac{f_y \cdot H}{h_{\text{px}}}, \quad \mathbf{x}^* = \mathbf{o} + d\hat{\mathbf{b}} \quad (3)$$

Distance-size ambiguity. The formula (3) prior assumption H for the target class. If there are two objects of the same class with known actual height $H_1 \neq H_2$, placed at two distances d_1, d_2 such that

$$\frac{H_1}{d_1} = \frac{H_2}{d_2} \left(= \frac{h_{\text{px}}}{f_y} \right), \quad (4)$$

Two objects show the same h_{px} on image and the formula (3) does not distinguish — this is distance-size ambiguity which is typical of monocular camera, it is not solved by only formula, geometric or deep learning model (Section V-A). Further analysis of why Mapanything’s depth inference does not automatically overcome this limitation. The clustering step DBSCAN at S6, prioritizing the densest and persistent point cluster across multiple frames, reducing the risk of confusing it with false objects that appear only briefly, but does not eliminate confusion between two same class entities — the method implicitly assumes that the environment contains only one instance of the target class, which is consistent with the current experimental setup (Section V-C).

None of quantities above depend on the pose odometry of a single frame, except for $\mathbf{o}$ (reference results) — the alignment uses entire trajectory, while the distance estimation does not use odometry. Figure 1 is a visual overview of entire localization pipeline that described above, include three blocks (i) 3D reconstruction, (ii) Attaching YOLO semantic labels, and (iii) distance estimation.

IV. EXPERIMENTAL RESULTS

With each capture session, three processing blocks at Section III-D are performed sequentially: (i) Reconstructing a 3D map using MapAnything, (ii) Attaching YOLO labels on each 3D point to have semantic point cloud, and (iii) Estimating single-image distance — two orient signals (from (ii)) and distance (from (iii)) then combined to create localization. The method is validated at six independent capture session at 3.00 m (measured using a measuring tape.), Each time, the robot observes the plant pot from a different trajectory. To

evaluate the stability when extending the range, the process is repeated at 5 m and 7 m, five sessions at each level, combining with 3 m at Table II and analyzed separately at Section IV-D. Figure 2 illustrates the experimental environment from a single scan of the entire room. (detail at Section IV-C).

A. Quantitative results

Table II The results at each session are summarized, together with detection/clustering quality indicators : the YOLO detection confidence at frame which used to estimate distance, the height of bounding box respectively, the number of points which in the largest semantic cluster used to inference orientation.

Six captured sessions show errors in the range 2–9% compared to real-world values; no session shows an excessively large error 0.26 m. Notably, the YOLO confidence varies considerably (0.63–0.90) while the number of semantic cluster points varies at high level ($>29\,700/30\,000$ points, meaning target cluster accounts for more than 99% of the sampled points) — this shows that the objects are reconstructed by MapAnything into a contiguous point cluster, with little fragmentation, regardless of the detection confidence.

Correlation between input quality indicators and localization error The Pearson correlation coefficient between YOLO confidence and the absolute error is $r = 0.34$; between the number of clusters and the error $r = 0.20$ — Both correlations are weak and statistically insignificant. with $n = 6$. This result is consistent with the theoretical expectation. At section V-A: the accuracy mainly comes from separating two signals from pose odometry each frame, rather than coming from high-confidence detection or large semantic cluster.

B. Visualization on the point map

Figure 6 illustrates localized object positions provided with the robot trajectory on the point cloud map of six captured sessions. Figure 5 visualizes the distribution of estimated distances of six sessions around the ground truth measurements 3.00 m, showing that the estimates converge around the reference line within the range ± 1 standard deviation. The results of six sessions show average errors of about 6%, not clearly depending on trajectory shape (Figure 6) or the 2D detection confidence (Table II) — consistent with the argument presented in Section V-A. Figure 4 Comparison of the real-world experimental environment with a YOLO11n frame detecting the plant pot.

C. Extension: Full-room semantic map scanning

To evaluate scalability of the pipeline beyond the single target localization scenario, we move the robot around the room (239 frames, instead of the 130 frames focused on a single object as in Section IV) and run the entire pipeline again for S1–S8 unlimited target semantic classes. The result is a 3D map with 38.8 million points, while YOLO11n attaches 12 different COCO labels. Figure 2 compares a real-world photograph of the room with the top-down view of the point map.

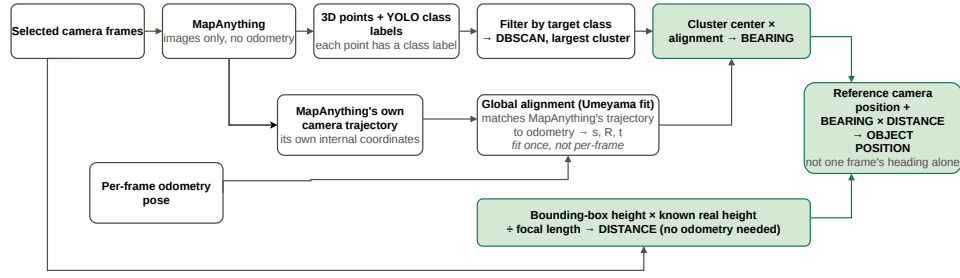


Fig. 1: Object localization pipeline combining bearing estimated from MapAnything semantic point clusters with monocular distance estimation.



(a) Real-world captured image



(b) Point map (top view, aligned with the direction of the real-world image))

Fig. 2: Full-room scan: real-world image (left) and 3D point map reconstructed from 239 frames (right). Yellow marker: centroid of the largest point cluster; red line: robot trajectory.

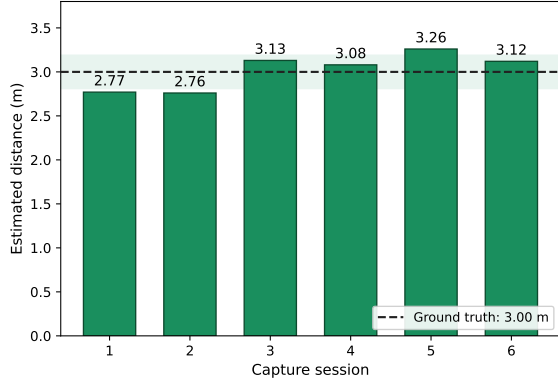
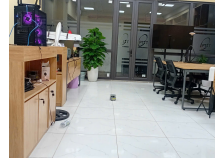


Fig. 3: Estimated distances across six capture sessions, compared with the ground-truth distance of 3.00 m (dashed line). Shaded region: ± 1 standard deviation around the mean.

The reconstructed map accurately preserves the overall shape and layout of the furniture (tables, chairs, cabinets, and the plant pot at their corresponding relative positions). However, due to the QVGA camera combined with the YOLO11n nano model, some detected labels are false positives—for example, “car”, “airplane”, and “traffic light”, which are largely absent from the actual office environment—consistent with the hardware limitations discussed in Section III-B. These results demonstrate that the pipeline generalizes well to full-room mapping, although the reliability of semantic labeling remains constrained by the current sensing hardware.



(a) Experimental environment



(b) YOLO11n plant pot detection

Fig. 4: Real-world experimental environment (left): the Hawkbot robot positioned in the middle of the floor in an office space, with the plant pot as the target object. YOLO11n detection on an ESP32-CAM frame (right), with a confidence score of 0.75.

D. Distance-dependent accuracy evaluation

The six capture sessions in Section IV were all conducted at 3.00 m. To evaluate whether the method remains accurate at greater target distances, we repeated the B1–B8 procedure, keeping $H = 1.25$ m and using the same target plant pot, at two additional distances: five sessions at 5 m and five sessions at 7 m, with the results summarized in Table II.

The three result groups reveal two distinct error patterns. At 3 m and 5 m, the mean error nearly cancels (+0.6% and −1.5%), but the standard deviations are relatively large (0.19 m, 0.22 m), with individual sessions deviating up to $\pm 10\%$ — characteristic of random noise. At 7 m, in contrast, the standard deviation is very small (0.07 m, less than one-third of the two shorter ranges), yet all five sessions consistently show a −11% bias, characteristic of a systematic

TABLE II: Plant localization results at three real-world distances (3 m: six capture sessions; 5 m and 7 m: five sessions each), together with input quality indicators

Real-world distance	Capture session	Estimated distance	Error w.r.t. ground truth	YOLO confidence	BBox height (px)	Semantic cluster points
3 m	1	2.77 m	−7.6%	0.63	192.9	29 841
	2	2.76 m	−8.1%	0.90	193.9	29 992
	3	3.13 m	+4.2%	0.78	170.9	29 974
	4	3.08 m	+2.6%	0.65	173.6	29 773
	5	3.26 m	+8.5%	0.76	164.2	29 913
	6	3.12 m	+3.9%	0.76	171.4	30 000
Mean		3.02 m	Std. dev. 0.19 m, mean error +0.6% (absolute mean 6.2%)			
5 m	1	4.49 m	−10.2%	0.81	119.1	29 606
	2	5.07 m	+1.4%	0.74	105.4	29 984
	3	5.00 m	+0.0%	0.85	106.9	25 826
	4	5.06 m	+1.2%	0.82	105.6	25 932
	5	5.01 m	+0.3%	0.75	106.6	22 543
Mean		4.93 m	Std. dev. 0.22 m, mean error −1.5% (absolute mean 2.6%)			
7 m	1	6.21 m	−11.3%	0.85	86.1	29 742
	2	6.22 m	−11.1%	0.51	85.9	29 588
	3	6.17 m	−11.9%	0.73	86.6	30 000
	4	6.34 m	−9.4%	0.71	84.3	29 662
	5	6.13 m	−12.4%	0.82	87.2	29 933
Mean		6.21 m	Std. dev. 0.07 m, mean error −11.2%			

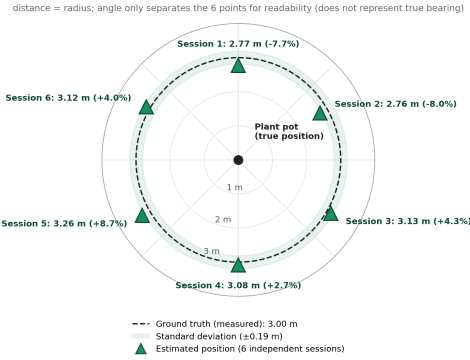


Fig. 5: Distribution of estimated distances across six independent capture sessions around the ground-truth distance of 3.00 m (dashed circle). Green triangles denote the estimated distance for each session, the black dot at the center indicates the true plant-pot position, and the shaded region represents the ± 1 standard deviation range (0.19 m). The angular positions are used only to separate the points for readability and do not represent actual bearing directions.

error. Because $d = f_y H / h_{px}$ gives a proportional relationship between d and $1/h_{px}$, a constant relative error in f_y (camera calibration) or H (assumed plant-pot height) produces exactly the observed nearly-constant bias. Recalibrating f_y or measuring H more accurately is therefore a direct mitigation (Section V-C).

TABLE III: Mean error: monocular distance vs. direct MapAnything 3D position

Distance	Single-image	3D position
Ground truth	(in paper)	(direct)
3 m	+0.6%	−21.9%
5 m	−1.5%	−22.2%
7 m	−11.2%	−38.4%

V. DISCUSSION

A. Why Global Fitting Is More Stable Than Per-Frame Triangulation

Both signals used by the method avoid dependence on the odometry pose of any single frame. The Umeyama alignment uses the entire camera trajectory, reducing the effect of local pose noise, while the monocular distance depends only on the object’s image size and the calibrated camera intrinsics.

Empirical validation. To evaluate whether the direct 3D position from MapAnything could replace the monocular distance, we compute $\|(s^* R^* \mathbf{c}_{\text{raw}} + \mathbf{t}^*) - \mathbf{o}\|$ for all sixteen sessions without using Eq. (3). The results are compared with the proposed method in Table III.

Directly using the 3D position from MapAnything produces substantially larger errors than the monocular formulation, with one case reaching nearly −80%. The Umeyama alignment fits a single (s, R, t) over the whole trajectory and cannot guarantee point-level accuracy, especially when rotational segments distort the geometry. Moreover, MapAnything’s depth estimation does not resolve the distance–size ambiguity —

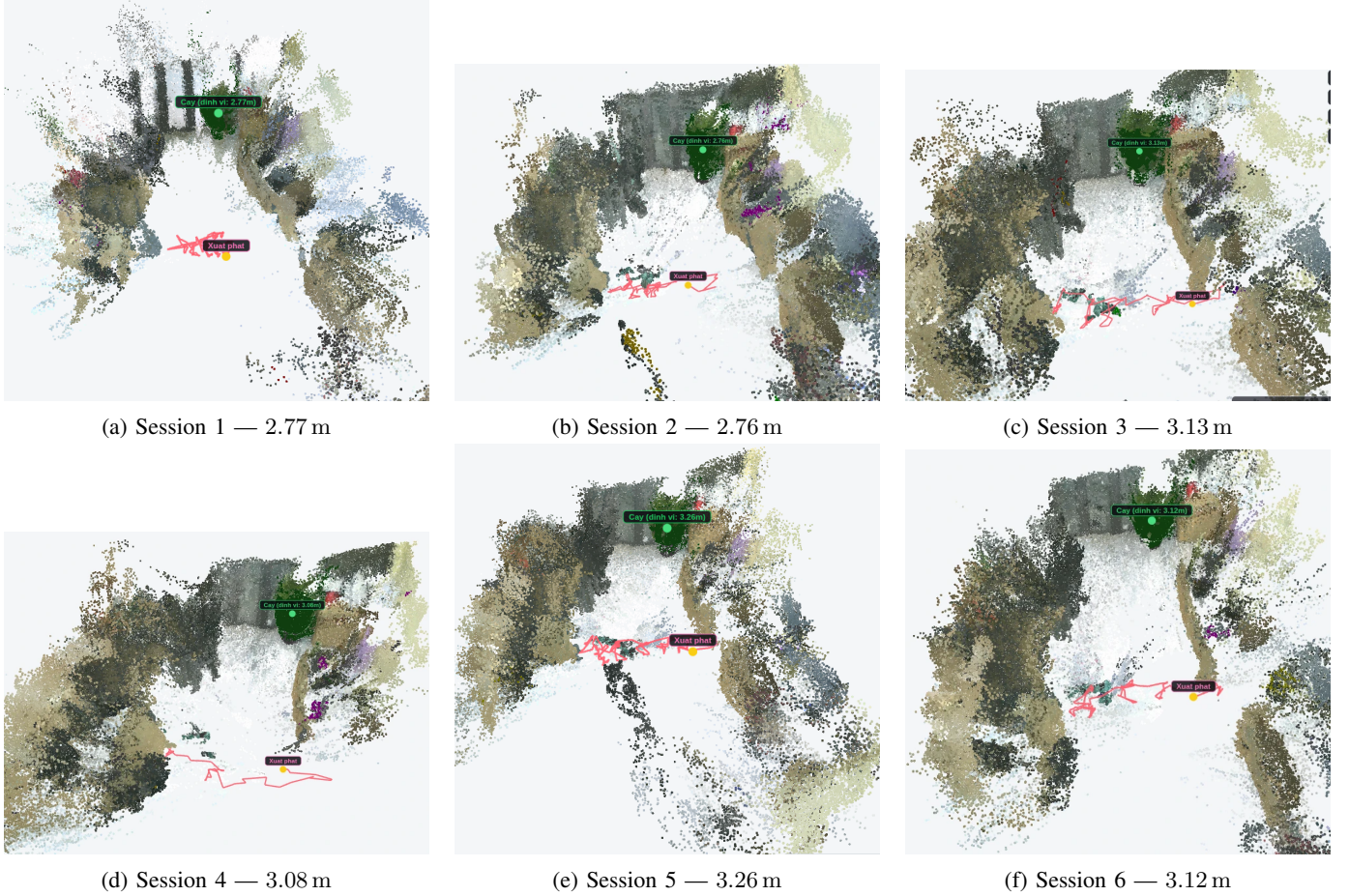


Fig. 6: Localized plant-pot position (green marker) and robot trajectory (red line, starting point marked) on the MapAnything point-cloud map for six independent capture sessions.

it merely shifts the metric reference from the known H to odometry. We therefore use the 3D position only for bearing, which is less sensitive to global scale errors, while monocular geometry provides the absolute distance.

B. Hardware Alternatives Tested

- **ArUco:** A fixed 15 cm marker was tested as an anchor. With the QVGA camera, `solvePnP` produced errors of 0.3 m to 0.7 m beyond 2 m or under oblique views, comparable to or larger than the drift to be corrected.
- **EKF (/odometry/filtered):** An EKF combining IMU measurements was tested, but its covariance diverged to an unrealistic magnitude ($\sim 10^{22}$), making it unsuitable without further tuning.

C. Limitations

The study is based on sixteen capture sessions (six at 3 m, five at 5 m, and five at 7 m) in a single room with one target type (plant pot), which limits the assessment of generalization. The monocular distance estimate requires the true object height to be known, making it less suitable for objects with varying dimensions. The systematic -11% bias at 7 m suggests that H or f_y should be calibrated more accurately

for longer ranges. Global alignment also depends on sufficient trajectory diversity; short or nearly straight trajectories may reduce its reliability. Future work should evaluate more object types, capture sessions, and environments, while higher-resolution cameras and greater edge-computing capacity could enable more robust solutions.

VI. CONCLUSION

We presented an absolute object-localization method in a semantic 3D map for an autonomous delivery robot, combining bearing estimated from semantic point clusters through global trajectory alignment with monocular distance from a single frame, without relying on odometry. No individual step requires an accurate pose from a single frame. Across sixteen capture sessions at 3 m, 5 m, and 7 m, the method achieved mean errors below 2% at the two shorter ranges, while showing a systematic bias of approximately -11% at 7 m. This indicates the need for more accurate calibration of the assumed object height or camera focal length when extending the operating range. Separating bearing estimation from distance estimation provides an effective localization design for low-cost robotic platforms.

REFERENCES

- [1] R. Mur-Artal, J. M. M. Montiel, and J. D. Tardós, “Orb-slam: A versatile and accurate monocular slam system,” *IEEE Transactions on Robotics*, vol. 31, no. 5, pp. 1147–1163, Oct 2015.
- [2] J. McCormac, A. Handa, A. Davison, and S. Leutenegger, “Semanticfusion: Dense 3d semantic mapping with convolutional neural networks,” in *2017 IEEE International Conference on Robotics and Automation (ICRA)*, May 2017, pp. 4628–4635.
- [3] A. Rosinol, M. Abate, Y. Chang, and L. Carlone, “Kimera: an open-source library for real-time metric-semantic localization and mapping,” in *2020 IEEE International Conference on Robotics and Automation (ICRA)*, May 2020, pp. 1689–1696.
- [4] C. Cadena, L. Carlone, H. Carrillo, Y. Latif, D. Scaramuzza, J. Neira, I. Reid, and J. J. Leonard, “Past, present, and future of simultaneous localization and mapping: Toward the robust-perception age,” *IEEE Transactions on Robotics*, vol. 32, no. 6, pp. 1309–1332, Dec 2016.
- [5] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016, pp. 779–788.
- [6] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics yolo11,” 2024. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [7] S. Ren, K. He, R. Girshick, and J. Sun, “Faster r-cnn: Towards real-time object detection with region proposal networks,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 39, no. 6, pp. 1137–1149, June 2017.
- [8] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, “End-to-end object detection with transformers,” in *Proc. ECCV*, 2020.
- [9] Meta AI (FAIR), “MapAnything: Universal feed-forward 3d reconstruction,” 2025. [Online]. Available: <https://huggingface.co/facebook/map-anything>
- [10] L. Yang, B. Kang, Z. Huang, X. Xu, J. Feng, and H. Zhao, “Depth anything: Unleashing the power of large-scale unlabeled data,” in *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2024, pp. 10 371–10 381.
- [11] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, “Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 3, pp. 1623–1637, March 2022.
- [12] S. Umeyama, “Least-squares estimation of transformation parameters between two point patterns,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 4, pp. 376–380, April 1991.
- [13] P. Besl and N. D. McKay, “A method for registration of 3-d shapes,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239–256, Feb 1992.
- [14] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proc. KDD*, 1996.
- [15] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, E. Wheeler, and A. Y. Ng, “ROS: an open-source robot operating system,” in *ICRA Workshop on Open Source Software*, 2009.
- [16] S. Garrido-Jurado, R. Muñoz-Salinas, F. Madrid-Cuevas, and M. Marín-Jiménez, “Automatic generation and detection of highly reliable fiducial markers under occlusion,” *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0031320314000235>